

Práctica 3 Adquisición Forense.

tp: https://catedras.linti.unlp.edu.ar/pluginfile.php/152005/mod_resource/content/1/TP3.pdf

Adquisición Forense

Los siguiente ejercicios (retos del CTF) son de carácter obligatorio para cumplir con esta entrega:

Ejercicio - Sectores - Práctica 3

Obtener la cantidad de sectores de la imagen pen.dd

para ver la cantidad de sectores usar comando fdisk -l (Este comando se utiliza para analizar la estructura de particiones de un disco o una imagen)

```
(kali㉿kali)-[~/Downloads]
$ fdisk -l pen.dd
Disk pen.dd: 512 MiB, 536870912 bytes, 1048576 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x00000000
```

Disk pen.dd: 512 MiB, 536870912 bytes, 1048576 sectors
Rta IFD{1048576}

Ejercicio - Hash JPG - Práctica 3

Hash SHA1 del archivo "mario.jpg" contenido en pen.dd

- 1ero crear un punto de montaje, es decir una carpeta vacia para acceder al contenido de la imagen [pen.dd](#)
- 2do montar la imagen con el comando mount para conectar la imagen [pen.dd](#) a la carpeta que acabamos de crear. Usamos la opcion loop para tratar el archivo de imagen como si fuera un disco fisico y ro para montarlo en modo solo lectura (read only), para no alterar la evidencia (paso fundamental en forensia)
 - sudo mount -o ro,loop pen.dd /mnt/pendrive
- 3ero encontrar el archivo
 - ls -l /mnt/pendrive
- 4to calcular el SHA1
 - sha1sum /mnt/pendrive/mario.jpg

```
(kali㉿kali)-[~/Downloads]
$ sha1sum /mnt/pendrive/mario.jpg
77d5cc89cf13464a0ed4671f755854e8830903ae /mnt/pendrive/mario.jpg
```

Rta IFD{77d5cc89cf13464a0ed4671f755854e8830903ae}

Ejercicio - Imagen del server - Práctica 3

Se les proporcionó un archivo de disco de máquina virtual (vmdk). Tienen que calcular el hash MD5 de la imagen raw (dd) de ese archivo vmdk

Lo descargamos, hacemos la verificación de integridad con MD5 y lo comparamos con el del enunciado y vemos que es distinto

```
(kali㉿kali)-[~/Downloads]
$ md5sum vm-104-disk-0.vmdk
d41d8cd98f00b204e9800998ecf8427e  vm-104-disk-0.vmdk
```

enunciado: MD5 de verificación: a068769e82aa4aba7d24441db0284dd9
vm-104-disk-0.vmdk

El hash que tuvimos d41d8cd98f00b204e9800998ecf8427e es un hash MD5 de un archivo vacío y para chequear esto hacemos → ls -lh vm-104-disk-0.vmdk

```
(kali㉿kali)-[~/Downloads]
$ ls -lh vm-104-disk-0.vmdk
-rw-r--r-- 1 kali kali 0 Oct 25 17:32 vm-104-disk-0.vmdk
```

y si muestra que tiene 0 bytes.

Borre y volví a descargar el archivo, fue problema de la descarga, no es parte del desafío. Ahora si, tenemos el mismo MD5 que el desafío.

```
(kali㉿kali)-[~/Downloads]
$ md5sum 'vm-104-disk-0(1).vmdk'
a068769e82aa4aba7d24441db0284dd9  vm-104-disk-0(1).vmdk
```

El siguiente paso es convertir la imagen VMDK a RAW (.dd)

Para esto vamos a usar la herramienta qemu-img

Vamos a usar este comando: qemu-img convert -f vmdk -O raw 'vm-104-disk-0(1).vmdk'

[salarios.dd](#)

qemu-img convert: Le dice al programa que queremos convertir una imagen.

-f vmdk: Especifica que el formato del archivo de entrada es vmdk.

-O raw: Especifica que el formato de salida (Output) que queremos es raw (o dd).

'vm-104-disk-0(1).vmdk': Es nuestro archivo de origen verificado.

salarios.dd: Es el nombre que le daremos al nuevo archivo de imagen raw.

Según la gente de informática del organismo, se había realizado una captura del tráfico de red del servidor de Salarios y necesitan recuperarla.

Deben calcular el hash MD5 de dicha captura.

Descargamos qemu-utils

convertimos la imagen VMDK a RAW (dd):

- **qemu-img convert -f vmdk -O raw 'vm-104-disk-0(1).vmdk' salarios.dd**

qemu-img convert: Le dice al programa que queremos convertir una imagen.

-f vmdk: Especifica que el formato del archivo de entrada es vmdk.

-O raw: Especifica que el formato de salida (Output) que queremos es raw (o dd).

'vm-104-disk-0(1).vmdk': Es nuestro archivo de origen, ya verificado.

salarios.dd: Es el nombre que le daremos al nuevo archivo de imagen RAW.

```
(kali@kali)-[~/Downloads]
$ qemu-img convert -f vmdk -O raw 'vm-104-disk-0(1).vmdk' salarios.dd
```

Calculamos el Hash MD5 de la imagen [salarios.dd](#) creada anteriormente.

```
(kali@kali)-[~/Downloads]
$ md5sum salarios.dd
2af57b5988f6d17d805acb1bf3ee3412 salarios.dd
```

Rta: IFD{2af57b5988f6d17d805acb1bf3ee3412}

Ejercicio - Captura de red - Práctica 3

Según la gente de informática del organismo, se había realizado una captura del tráfico de red del servidor de Salarios y necesitan recuperarla. Deben calcular el hash MD5 de dicha captura.

Para ver la captura tenemos que entrar a la imagen de [salarios.dd](#) que creamos en el desafío anterior para eso usamos → `sudo mkdir /mnt/salarios`
paso 2 montamos la imagen del disco ([salarios.dd](#)) en la carpeta que creamos. Esto es para poder explorarla como si fuera un pendrive. Usamos la opción ro (read only) para montarla en modo solo lectura, para no alterar nada. → `sudo mount -o ro,loop salarios.dd /mnt/salarios`

```
(kali@kali)-[~/Downloads]
$ sudo mount -o ro,loop salarios.dd /mnt/salarios
mount: /mnt/salarios: wrong fs type, bad option, bad superblock on /dev/loop0, missing dmesg(1) may have more information after failed mount system call.
```

¿Qué pasó?

El error `wrong fs type, bad option, bad superblock` significa que el comando `mount` no pudo entender qué tipo de sistema de archivos hay en la imagen.

La razón es que `salarios.dd` no es una partición, es una imagen de un disco entero. Imagina que `salarios.dd` es una caja grande que contiene una o más cajas más pequeñas (las particiones) adentro. Estás tratando de abrir la caja grande y ver el contenido directamente, pero lo que tenés que hacer es abrir la caja pequeña que está adentro, que es la que realmente contiene los archivos.

¿Cómo lo solucionamos?

Necesitamos decirle al comando mount en qué punto exacto dentro de la imagen salarios.dd comienza la partición que queremos montar. Para eso, primero tenemos que averiguarlo

Paso 1: Analizar la tabla de particiones

Vamos a usar la herramienta fdisk para que nos muestre el "mapa" del disco y nos diga dónde empieza la partición.

Ejecutá el siguiente comando en tu terminal → `fdisk -l salarios.dd`

Este comando te mostrará una tabla con las particiones. Prestá atención a la columna que dice Start (Inicio). Ese número nos dice en qué sector comienza la partición

```
(kali㉿kali)-[~/Downloads]
$ fdisk -l salarios.dd
Disk salarios.dd: 10 GiB, 10737418240 bytes, 20971520 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xf79019dd

Device      Boot    Start        End    Sectors   Size Id Type
salarios.dd1 *        2048    14682111    14680064    7G 83 Linux
salarios.dd2             14684158    20969471     6285314    3G  5 Extended
salarios.dd5             14684160    20969471     6285312    3G 82 Linux swap / Solaris
```

Análisis del "Mapa" del Disco

Lo que fdisk te está mostrando es el mapa de particiones dentro de salarios.dd. Vemos tres entradas:

- salarios.dd1: Esta es la partición principal de Linux (Tipo 83). Es la más grande (7G) y es casi seguro que contiene los archivos que buscamos.
- salarios.dd2: Es una partición "Extendida", que solo sirve como contenedor para otras particiones.
- salarios.dd5: Es la partición de swap, la memoria virtual de Linux. Aquí no encontraremos archivos de usuario.

La pieza de información clave que nos da este mapa es para salarios.dd1:

- Start (Inicio): 2048
- Units (Unidades): sectors of 1 * 512 = 512 bytes

Esto nos dice que la partición que nos interesa comienza en el sector 2048, y que cada sector mide 512 bytes.

Paso 2.1: Montar la Partición con el Offset Correcto

Ahora podemos darle al comando mount la dirección exacta de dónde empezar a leer. Lo hacemos calculando un offset (desplazamiento).

Offset = Sector de Inicio * Tamaño del Sector Offset = 2048 * 512

Vamos a usar una sintaxis especial para que la terminal haga este cálculo por nosotros.

1. Asegurate de estar en el directorio ~/Downloads.

Ejecutá el siguiente comando mount corregido:

Bash

```
sudo mount -o ro,loop,offset=$((2048*512)) salarios.dd /mnt/salarios
```

- 2.

- `offset=$((2048*512))`: Esta es la parte nueva. Le dice a mount: "No empieces a leer desde el principio del archivo, sino que saltá 2048 * 512 bytes y empezá a leer desde ahí".

Nos dios esto

```
(kali@kali)-[~/Downloads]
$ sudo mount -o ro,loop,offset=$((2048*512)) salarios.dd /mnt/salarios
[sudo] password for kali:
mount: /mnt/salarios: cannot mount /dev/loop0 read-only.
dmesg(1) may have more information after failed mount system call.
(kali@kali)-[~/Downloads]
```

¿Qué Pasó?

El nuevo error `cannot mount /dev/loop0 read-only` significa que el sistema de archivos dentro de la imagen (`salarios.dd1`) no fue "desmontado" limpiamente la última vez que se usó. Está marcado como "sucio" (`dirty`).

Cuando Linux ve un sistema de archivos en este estado, por seguridad, quiere "repararlo" reproduciendo algo llamado `journal` antes de montarlo. Esta operación de reparación necesita permisos de escritura, lo que choca directamente con nuestra orden de montarlo en modo solo lectura (`ro`). Como no puede hacer ambas cosas, te da un error.

¿Cómo lo Solucionamos?

Vamos a añadir una opción especial al comando `mount` para decirle: "Sé que el disco no se cerró bien, pero montalo en modo solo lectura de todas formas e ignora el `journal` por ahora".

La opción mágica es `noload`.

Montar la Partición Ignorando el Journal → `sudo mount -o ro,loop,noload,offset=$((2048*512)) salarios.dd /mnt/salarios`

Paso 3: Encontrar la captura y calcular su hash

Ahora que la partición está montada correctamente, hacemos → `ls -lR /mnt/salarios`

1. (Usé `-lR` para que liste todo, incluyendo el contenido de las subcarpetas, y así sea más fácil encontrar el archivo).
2. Identifica la captura: Busca un **archivo** que parezca una **captura de red** (probablemente con extensión **.pcap** o similar).

La salida me devolvio muchísimos archivos como resultado así que vamos a filtrar usando → `ls -lR /mnt/salarios | grep -i '\.pcap'`

Me dio una lista mucho mas chica. Pero no cumplen con la extensión `.pcap`. Se encontro uno pero termina `.pcap.xml` que no nos sirve, no es lo que estamos buscando. Vamos a probar el comando **`sudo find/mnt/salarios -type f -iname "*.pcap"`**

- `sudo`: Ejecuta el comando como superusuario para que no tengamos más errores de "Permission denied".
- `find`: El comando para buscar.
- `/mnt/salarios`: El directorio donde queremos empezar a buscar.
- `-type f`: Le dice a find que busque solo ficheros (archivos), no directorios.
- `-iname "*.pcap"`: Esta es la condición de búsqueda. `-iname` busca por nombre sin distinguir mayúsculas de minúsculas, y `*.pcap` significa "cualquier archivo que termine en `.pcap`".

devolverá la ruta exacta del archivo de captura si existe.

```
(kali㉿kali)-[~/Downloads]
$ sudo find /mnt/salarios -type f -iname "*.pcap"
/mnt/salarios/root/salarios.pcap

(kali㉿kali)-[~/Downloads]
$ md5sum /mnt/salarios/root/salarios.pcap
md5sum: /mnt/salarios/root/salarios.pcap: Permission denied
```

El error "Permiso denegado" apareció porque el archivo salarios.pcap está en el directorio /mnt/salarios/root. En Linux, la carpeta /root es el directorio personal del superusuario y está protegida. Tu usuario normal (kali) no tiene permiso para leer archivos allí. Solucion:

sudo md5sum /mnt/salarios/root/salarios.pcap

3. Calcula el hash MD5:

md5sum /mnt/salarios/home/usuario/captura.pcap

```
(kali㉿kali)-[~/Downloads]
$ sudo find /mnt/salarios -type f -iname "*.pcap"
/mnt/salarios/root/salarios.pcap

(kali㉿kali)-[~/Downloads]
$ md5sum /mnt/salarios/root/salarios.pcap
md5sum: /mnt/salarios/root/salarios.pcap: Permission denied

(kali㉿kali)-[~/Downloads]
$ sudo md5sum /mnt/salarios/root/salarios.pcap
d60bc7ebe3d48b9ffaafc26924a7d7188 /mnt/salarios/root/salarios.pcap
```

Rta: IFD {d60bc7ebe3d48b9ffaafc26924a7d7188}